MIOPE: A Modular framework for Input and Output Privacy in Ensemble inference

Kyrian Maat[1], Gareth T. Davies[2], Zoltán Ádám Mann[3], Joppe W. Bos[2] and Francesco Regazzoni[1,4]

[1] University of Amsterdam, Amsterdam, Netherlands
[2] NXP Semiconductors, Leuven, Belgium
[3] University of Münster, Institut für Informatik, Münster, Germany
[4] Università della Svizzera italiana, Faculty of Informatics, Lugano-Viganello, Switzerland

Abstract. We introduce a simple yet novel framework for privacy-preserving machine learning inference that allows a client to query multiple models without a trusted third party aggregator by leveraging homomorphically encrypted model evaluation and multi-party computation. This setting allows for dispersed training of models such that a client can query each separately, and aggregate the results of this 'ensemble inference'; this avoids the data leakage inherent to techniques that train collectively such as federated learning. Our framework, which we call MIOPE, allows the data providers to keep the training phase local to provide tighter control over these models, and additionally provides the benefit of easily retraining to improve inference of the ensemble. MIOPE uses homomorphic encryption to keep the querying client's data private and multi-party computation to hide the individual model outputs. We illustrate the design and trade-offs of input- and output-hiding ensemble inference as provided by MIOPE and compare performance to a centralized approach. We evaluate our approach with a standard dataset and various regression models and observe that the MIOPE framework can lead to accuracy scores that are only marginally lower than centralized learning. The modular design of our approach allows the system to adapt to new data, better models, or security requirements of the involved parties.

Keywords: Privacy-enhanced Machine Learning · Fully Homomorphic Encryption · Security for Machine Learning

1 Introduction

Machine Learning has found increasing success in finding patterns that humans are not able to decipher, such as in diseases or DNA [GST+99, AAR+19]. However, to correctly find and act upon these patterns requires a large amount of high-quality data for training [DDS+09]: in many settings collecting such data at a central point is infeasible due to legal and ethical safeguards that protect individual privacy or commercial interests. Even if one or more models can be trained, if the input to the model(s) is very sensitive then similar legal/ethical boundaries may prohibit cleartext (transmission and) processing of queries (inputs) or availability of the resulting insights (outputs) during inference [GLMS23].

This research has received funding from the European Union's Horizon Research and Innovation Actions under the Grant Agreement No. 101095717 (SECURED).

E-mail: k.maat@uva.nl (Kyrian Maat), garethhthomas.davies@nxp.com (Gareth T. Davies), zoltan.mann@uni-muenster.de (Zoltán Ádám Mann), joppe.bos@nxp.com (Joppe W. Bos), f.regazzoni@uva.nl (Francesco Regazzoni)

In scenarios with large volumes of horizontally partitioned data that cannot be centralized, there are two distinct avenues: keeping the data in their silos and performing ensemble learning on multiple models, or training a joint model on the datasets in a way that provides some degree of data input privacy. This second approach is often realized via Federated Learning (FL) [KMR15, KMY+16], which allows for multiple data owners to collectively train a single model by sharing model weights of local models to contribute to a single centralized model. Multiple rounds of communication are required to reach convergence but the final result is built upon the data of the local models, and this can achieve similar results to a non-privacy preserving centralized training [ZLL+18].

While Federated Learning enables the creation of a (potentially high quality) single model, it also has clear drawbacks. Most FL set-ups require a centralized aggregator, often a third-party server, to be able to update the models and converge. Even in a setting without a server, a specific participant needs to be chosen to do these aggregations[1]. This introduces a singular node of failure, and additionally adds the requirement of extra privacy-preserving layers such as differential privacy [Dwo06] or secure aggregation [BIK+17] that have to be ensured to minimize leakage of data to the participants/model server. If a data owner acquires more training data after the FL training process finished, then the process would need to either start again, or a dedicated procedure would be required to incorporate the new data into the single model. Furthermore, one entity will have control of the global model at the time the training is finished. Data owners are often required by legal frameworks or policy to ensure that their data never leaves the organization or its premises, and model weights that may allow other parties to obtain even partial statistics on the training database could be covered by these restrictions. Model updates that leak during the training process of FL could even lead to more potent attacks, such as gradient inversion attacks [GBDM20, YZH22] where sensitive training data could be obtained. Another caveat is that if any participant in the FL process is found to be malicious or has incorrect data, then the integrity of the entire model will be in doubt, and a full re-training is often required. In many cases—such as for personally identifiable health care data—these privacy and security risks may mean that FL—even techniques such as decentralized FL [HBJ18]—would be seen as too risky in the eyes of regulation enforcers: the consequence is that the data remains in silos.

Ensemble inference[2] on the other hand—ensemble inference is the usage of inference over trained local models in an ensemble, where the model outputs are aggregated or summed on the entity querying the models—can be a useful approach in certain circumstances [ZDW13, ZLZ23, KTBG20]. However, performing ensemble inference allows clients to learn the output of multiple models directly via their input query. Obtaining the model outputs leaks significantly more information about the underlying models and their training data than a single model (trained in either a centralized or a federated manner) would.

In this work we aim to provide a modular privacy-preserving machine learning inference framework that enables analytics on data that is kept—by policy, legal framework or simply due to risk adversity—in distinct databases and cannot be shared; models are trained locally on these databases. Conceptually our design framework is very straightforward, combining techniques from ensemble inference, secure aggregation (of model outputs rather than weights) and homomorphic encryption to provide a querying client with an aggregated output value that reveals a minimal amount[3] about the individual model outputs. Despite

[1] Decentralized Federated Learning [HBJ18, ZYWL25] settings can aggregate over collections of nodes to circumvent the issue of requiring a co-ordinator, but this approach may not be suitable for all federated learning instantiations (for example due to requiring decentralized storage for model updates or using a blockchain for aggregator elections/integrity).

[2] We use this terminology to distinguish from the use of an ensemble of models to collectively and iteratively *train* a better model using boosting or bagging, often referred to as *ensemble learning* [Efr92].

[3] The data leakage is not necessarily zero: following much of the secure multi-party computation (MPC) literature, it is only what can be calculated from the aggregated output. We elaborate on this in detail later.

its simplicity, we believe that our work is the first to combine MPC for model output privacy and homomorphic encryption for query input privacy where queries are made to an ensemble of models, so called "output-hiding ensemble inference".

To achieve our system requirements we use public-key (fully) homomorphic encryption (FHE) for encrypting client queries and to enable the model owners to apply 'masks' to their output values before a multi-party aggregation protocol is carried out to provide the querying client with a single combined output. We demonstrate the utility of our approach by providing an implementation for tree-based inference that combines the CKKS [CKKS17] and FHEW [DM15] schemes in the `OpenFHE` library [BAB+22], the NIST randomness beacon [oST25] for mask generation, and a simple method to sum all masks to zero in the query output aggregation component.

1.1 Brief Related Work

A related line of work concerns private inference queries on tree-based models, and this is often referred to as private decision tree evaluation [WFNL16, TMZC17, KNL+19]. In such papers, the querying client and the model-holding server engage in a multi-round cryptographic protocol, using multi-party computation or homomorphic encryption or a combination of the two, to evaluate the tree-based model without leaking the client's input or the model's output to the server. While these dedicated interactive protocols can provide efficiency gains compared to our approach of generically using FHE for this step, this class of solutions requires significant setup costs and lacks flexibility, and furthermore many solutions leak the model's topology/structure to the querying client. Our work is more closely aligned with non-interactive private decision tree evaluation [TKK19, TBK20, ALR+22, CDPP22, MNLK23]. The modular nature of our framework would allow any of these constructions to be used in the encrypted inference component.

The closest related literature to ours is that of Giacomelli et al. [GJK+18], which also aims to provide input privacy and output privacy for querying an ensemble of tree-based models using homomorphic encryption. Their framework fixes a machine learning structure (random forests), and requires that the model owners encrypt their models and store them on a dedicated processing entity that also performs output aggregation. This allows them to use homomorphic encryption schemes, but at the cost of maintaining this central server entity which MIOPE avoids.

1.2 MIOPE

In a nutshell, our system requires a set of trained machine learning models that perform the same task, e.g. a regression task that predicts the risk of a specific type of cancer based on the environmental and/or genomic data of a patient. The client wants to keep their input query private, and the model owners want to hide their model architecture and their individual model outputs in order to reduce the efficacy of black-box attacks [SSSS17].

Our framework specifically aims to provide model updatability (models can be retrained at any time) and model agnosticity (any model owner can employ any model architecture) while still allowing a client to query over data spread over multiple entities.

The main idea behind MIOPE and output-hiding ensemble inference is improving the flexibility of the participants and models involved. The participants can train their model whenever they like and do not depend on other models in the system for training and participating in the system allowing them all to train simultaneously and separately from each other. An added benefit of this design decision is that the MIOPE system has strong adaptability and resilience for participants in the system, as corrupted models or incorrect models can easily be removed, and new models can be added to the ensemble of models. As long as the input and output of the system is agreed upon, any model can be added to the MIOPE system and will not need any retraining of a global model that might be

utilized in a centralized or federated learning setting. One can also consider the general model-agnosticism that MIOPE allows another perk of adaptability, as the models can be heterogeneous in form if they adhere to the pre-agreed output requirements from the system. Similarly, the resilience of the MIOPE system also benefits from the plug-and-play nature of the model participants. A faulty mode, such as a wrong calibration in measuring, or a rogue model owner can easily be removed from the pool of queried models, whereas alternative approaches, such as FL, will require some retraining of the central model to remedy this problem. Additionally, because the model owners are free to bring their own local models, training time is amortized over the multiple model owners and the system as a whole can start running earlier than systems with communication during the training phase. This implies a shorter time to the first inference of the system than a setting where training has to be done together. However, conversely, due to the set-up of MIOPE, the time for a single query is higher than Federated Learning. A single model being queried with a single communication round is cheaper than the MIOPE alternative of multiple model owners that each have to communicate with the client, making the weakest link the bottleneck of the inference.

Contributions. In this work we provide:

- a modular framework for inference on an ensemble of machine learning models that gives data input privacy for the client, and individual model output privacy and model architecture hiding for the model owners;
- an implementation for inference on tree-based models in `OpenFHE` using the fully homomorphic schemes CKKS [CKKS17] and FHEW [DM15] (which may be of independent interest);
- an end-to-end demonstration of our privacy-preserving inference pipeline using regressors trained with the California housing dataset [PB97].

2 Preliminaries

In this section we define some standard definitions and concepts that we will use in the MIOPE framework.

2.1 Fully Homomorphic Encryption

Definition 2.1. *A public-key (fully) homomorphic encryption scheme* $\mathsf{HE} = \{\mathsf{KeyGen}, \mathsf{Enc}, \mathsf{Eval}, \mathsf{Dec}\}$ *is defined by the following algorithms:*

- *Key generation:* $(\mathsf{pk}, \mathsf{sk}, \mathsf{evk}) \leftarrow \mathsf{KeyGen}(\lambda, \mathsf{params})$. *On input a security parameter* λ *and other parameters* params, KeyGen *outputs an encryption key* pk, *a decryption key* sk *and an evaluation key* evk.
- *Encryption:* $\mathsf{C} \leftarrow \mathsf{Enc}(\mathsf{pk}, \mathsf{m})$. *On input an encryption key* pk *and a plaintext message* m, Enc *outputs a ciphertext* C.
- *Evaluation:* $\mathsf{C}' \leftarrow \mathsf{Eval}(\mathsf{evk}, \mathsf{C}, \mathsf{f})$. *On input an evaluation key* evk, *a ciphertext* C *and a function* f, Eval *outputs a new ciphertext* C'.
- *Decryption:* $\mathsf{m} \leftarrow \mathsf{Dec}(\mathsf{sk}, \mathsf{C})$. *On input a decryption key* sk *and a ciphertext* C, Dec *outputs a plaintext message* m *(or an error symbol such as* $\perp$*).*

The function $\mathsf{f} : \mathcal{D} \mapsto \mathcal{R}$ belongs to some viable set of functions that the scheme supports, hence this function set can be seen as a parameter of HE. Our syntax uses a

generic argument params to reflect that in general it is necessary for the entity to know at least some details about the supported functions f, for example the multiplicative circuit depth of f.

In general, all known FHE schemes, including the first one based on lattices and proposed by Gentry [Gen09], depend on the addition of randomness—called *noise* or *error*—to the ciphertext. Each HE evaluation also adds noise to the ciphertext (growing slowly for addition, and very quickly for multiplication). However, there is an upper bound on the amount of noise tolerated, and if that bound is exceeded, decryption will result in failure. To reduce noise, the scheme includes an auxiliary process called *bootstrapping* that homomorphically evaluates the decryption function with an encrypted secret key (i.e. resetting the noise to that of a single fresh encryption). In general, bootstrapping is a very expensive procedure since it requires homomorphic evaluation of what is often a high complexity function: applications often therefore define the cryptographic parameters to enable evaluation of a class of functions without ever needing to bootstrap.

HE schemes are often well suited to specific tasks due to their underlying structure, for example BGV [BGV11] and BFV [FV12, Bra12] are suited to precise computations on integers, CKKS [CKKS17] is an approximate scheme which is more suited to floating point operations where small accuracy loss is tolerable, while TFHE [CGGI16, CGGI20] and FHEW [DM15] are examples of FHE schemes working on binary circuits and therefore are well suited to functions that can be represented as (small) look-up tables (these schemes also support relatively fast bootstrapping).

In machine learning inference it is often necessary to perform a combination of operations, for example SIMD multiplication by weights for a neural network linear layer or a tree-based model's tree output pooling function and non-linear operations for activation functions or the comparisons made in decision nodes. In the encrypted domain it is therefore desirable to transform ciphertexts directly from one scheme to another: this process of *scheme switching* [BGGJ20, LHH+21] is essentially a more involved form of bootstrapping where the target scheme's decryption operation is homomorphically evaluated.

HE schemes are implemented in a variety of open-source libraries. The `OpenFHE` [BAB+22, BBB+22] library is utilised in this work and carries implementations of all of the schemes mentioned above. Other libraries exist, including `Concrete` [Zam22b] (and its machine learning wrapper `Concrete ML` [Zam22a]) which only supports the TFHE scheme.

2.2 Encrypted Inference for Machine Learning

Our work considers outsourced private inference, meaning that a machine learning model is evaluated entirely in the encrypted domain by a service provider (hereafter referred to as the model owner, but in practice these entities could be separated). Using homomorphic encryption makes it possible that the inference query input and output values are not made available in plaintext to this model owner [MWCB24].

A model owner is in possession of a trained model M that maps input features, represented as a vector qry, to some output space. To prepare for encrypted inference queries, the model owner represents M as a function f that is appropriate for the homomorphic encryption scheme HE they wish to use for inference, extracts the necessary parameters params and sends them to potential querying clients. A client C can then run $(\mathsf{pk}, \mathsf{sk}, \mathsf{evk}) \leftarrow \mathsf{KeyGen}(\lambda, \mathsf{params})$ and send evk to the model owner.

In the online inference phase, C encrypts their input feature vector with the encryption key $\mathsf{C} \leftarrow \mathsf{Enc}(\mathsf{pk}, \mathtt{qry})$, and sends C to the model owner. The model owner then homomorphically evaluates their model on the input query by doing $\mathsf{C}' \leftarrow \mathsf{Eval}(\mathsf{evk}, \mathsf{C}, \mathsf{f})$, and sends C' back to C. The client can decrypt the received message to get $f(\mathtt{qry}) \leftarrow \mathsf{Dec}(\mathsf{sk}, \mathsf{C}')$ as desired.

Note that in this description the query is encrypted and the result is decrypted by the client, and therefore the encryption key pk does not need to be made public. Later on for

the output masking step we will need to enable the model owner to encrypt random values and homomorphically add them to the model output, therefore this `pk` will need to be transmitted alongside `evk`. The HE implementation in `OpenFHE` supports this 'public-key encryption' variant of HE, whereas it is not supported by `Concrete ML`, where only a symmetric mode is available.

2.3 Multi-Party Computation

Secure multi-party computation (MPC) enables a group of parties to perform collaborative computations. The system consists of *input parties* that provide inputs, *computing parties* that actually engage in the computation phase, and *output parties* that receive output. This abstraction captures settings where these sets could be fully disjoint, or where they could fully overlap. Defining security for MPC can be a challenging exercise but generally at a minimum we expect *input privacy* for at least some of the input parties, even for adversaries that see intermediate computations and the output. Participating entities can be *honest-but-curious* where they perform their role according to the protocol specification yet try to learn as much additional information as possible, or *malicious* where they actively manipulate messages to reveal additional information or force other parties to accept an incorrect result.

A formal syntax is similarly difficult to provide in full generality, but for our purposes it is not necessary since the separation of parties is straightforward and the functions being collaboratively computed are simple. Our goal is for the MPC component of our pipeline to be a modular block that can adapt to the required implementation environment and threat model, for instance to tolerate malicious behavior or participants that might drop out during execution.

2.4 Randomness Beacons

Our approach requires a set of parties to use a randomness source to give a seed that is used as an input to deterministically generate structured and correlated outputs. The properties we require from this randomness source are *unpredictability* (predicting the next output is hard even given prior outputs), *unbiasedness* (outputs are statistically close to uniform) and *public access* (all parties can easily access its outputs).

Rabin [Rab83] first formalized the concept of a verifiable public randomness beacon. Suggestions for instantiation have included financial data [CH10] and the Bitcoin blockchain [BCG15]. Demand is sufficient for NIST to run their own randomness beacon [oST25] and it is this service that we will use for our implementation.

We will additionally require a method for our set of parties to *extract* randomness from this source using a pseudorandom function such as HMAC-PRF or AES-CBC with zero strings for key and IV.

3 System design of MIOPE

In this section we define our MIOPE system in generality by introducing the constituent components. A graphical representation of the MIOPE system is given in Fig. 1. The core driving idea behind output-hiding ensemble inference is the usage of multiple models that are trained in a local manner by their respective data owners and thus isolated from each other to inherently gain privacy and security on the models. When performing inference, all models are queried and the outputs are combined via an output summation strategy to obtain the final result.

To hide the model inputs and outputs from the model owners and to hide the individual model outputs from the querying client, different building blocks are required, namely

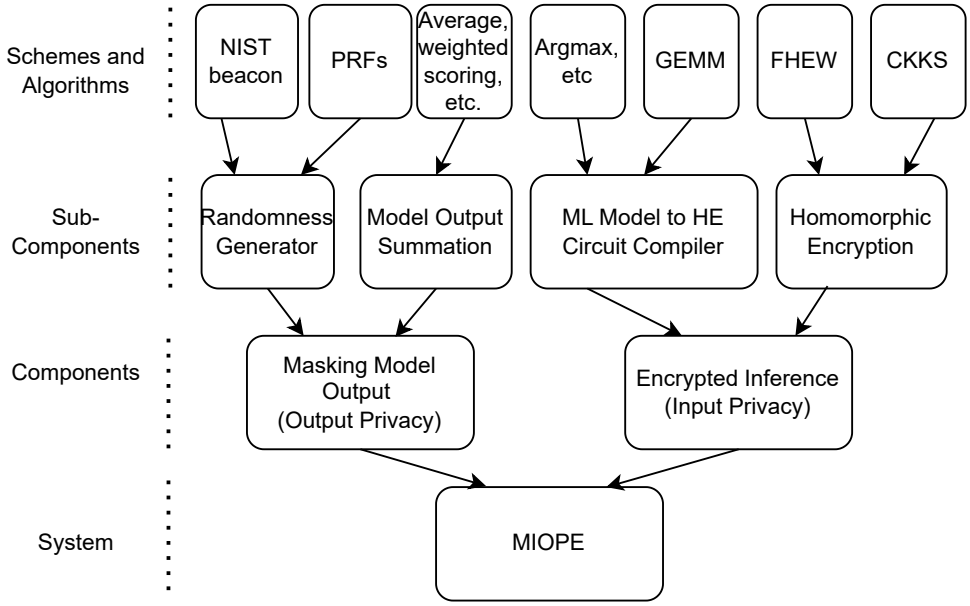

Figure 1: An illustration of the different components that are present in the realisation of the MIOPE system as a top-down approach. The top row represents our choices for our instantiation the modular sub-components in the second row.

encrypted inference and model output masking. The encrypted inference for our system uses homomorphic encryption and therefore requires a (usually straightforward) conversion of the trained machine learning models to circuits that are compatible with the evaluation function of the desired homomorphic encryption schemes, we explain this in more detail in Section 3.1. For masking the model outputs, we require both a randomness generator and model output summation, which we will discuss in Section 3.2. Lastly, the threat model that is associated with the MIOPE system is defined in Section 3.3.

3.1 Encrypted Inference in MIOPE

Encrypted inference for MIOPE requires homomorphic encryption (as introduced and defined in Section 2.2) to be applied over the multiple models. The usage of encrypted inference will achieve input privacy for the client, as the features used are homomorphically encrypted and the output is never able to be observed by the model owner. For a client to make queries to multiple models, they will need to run KeyGen once for each model and send the corresponding evaluation key to the model owner to enable future inference. Since the model owner only needs to transmit the multiplicative depth of their HE circuit to the querying client, they do not need to reveal their model architecture. Note that this KeyGen is for the scheme under which plaintexts are initially transmitted: if the model owner uses scheme switching to evaluate one or more parts of their model this fact is not revealed to the querying client.

The model owners can choose homomorphic encryption schemes independently of the querying client or of the other models in the system, therefore optimizing the encrypted inference pipeline based on the architecture of their model[4]. All we require in MIOPE

[4]As a reminder, the input to key generation is the multiplicative depth of the HE circuit being evaluated

is that the outputs of the models are normalized to the same space, for example for regressor models this could be $\{0.0, 0.1, \ldots, 100.0\}$. This requirement is to enable masking of outputs.

3.2 Model Output Masking

Encrypted Inference has given the system input privacy: none of the inputs that are given by the user are ever revealed to the model owners or any eavesdroppers. However, the client will decrypt all the model outputs locally. This means that the individual model output privacy is not guaranteed, allowing (adversarial) clients to potentially launch black-box attacks if it wishes to glean insights into the training data of the individual data silos.

MIOPE therefore incorporates model output masking to the encrypted outputs of each model to provide input and output privacy whilst providing the correct end results to the client. To perform model output masking, the outputs obtained from the encrypted inference need to be masked with random values, and since we do not have a dedicated and active trusted third party in our system we generically require a randomness source that is accessible by all model owners. Public randomness beacons—that precisely provide the functionality that we require—were introduced in Section 2.4.

To combine the model outputs, different strategies can be utilized depending on the data used in the model and the goal of the model, such as a classification, image recognition, or regression task.

The property we need from this procedure is that the model owners can take the outputs from a randomness source and turn them into values that 'cancel out' (for some definition that corresponds to the mathematical structure of the masks) when the client attempts to reconstruct a single output from the ensemble. As an example approach we will later describe the most straightforward solution for regressor models, namely masks that together sum to zero: summation of masked model outputs and division by the number of models then leads to the mean average of the regressor outputs.

The process for a model owner is therefore to homomorphically evaluate the model on the ciphertext query, retrieve the output of the randomness source that corresponds to the query, then to calculate the appropriate mask from this randomness source output[5]. For example, the mask generation process could take into account the time at which the client made their query and an identifier of that client. A (potentially non-interactive) protocol will then take place to turn these values into something that can (blindly) remove all masks when the client performs the reconstruction process. This process could be performed by the client on all of the encrypted-and-masked model outputs, or the client could first decrypt all of the masked model outputs and then perform an appropriate operation.

A generic syntax for a masking scheme that can be used for output-hiding ensemble inference is provided in Definition 3.1.

Definition 3.1 (Masking of Model Outputs)**.** *Let* $\mathsf{HE}_0 = \{\mathsf{KeyGen}_0, \mathsf{Enc}_0, \mathsf{Eval}_0, \mathsf{Dec}_0\}, \ldots,$ $\mathsf{HE}_{\mathsf{N}-1} = \{\mathsf{KeyGen}_{\mathsf{N}-1}, \mathsf{Enc}_{\mathsf{N}-1}, \mathsf{Eval}_{\mathsf{N}-1}, \mathsf{Dec}_{\mathsf{N}-1}\}$ *be* N *homomorphic encryption schemes. A client* $\mathtt{C}$ *has input* $\mathtt{qry}$ *and sends it in encrypted form to* N *model owners* $(M_0, \ldots, M_{\mathsf{N}-1})$*. After homomorphic processing the encrypted model output is* $c_i = \mathsf{Enc}_i(\mathsf{pk}_i, y_i)$ *where* $y_i = f_i(\mathtt{qry})$ *is the plaintext output of model* f_i*. The model owners then engage in a protocol* MASK.GET *such that each* M_i *receives or calculates a mask* r_i *and applies it to the encrypted output in a procedure* MASK.APPLY*, i.e. performs* $z_i = \mathsf{Eval}_i(\mathsf{evk}_i, c_i \bigodot r_i)$

(this is represented in generality by params in Def. 2.1). Therefore if all model owners use models of the same depth then the client could run KeyGen just once and send the same evaluation key material to all model owners, but this is not required by our approach.

[5]Conceptually, the model-output privacy-preserving techniques will be applied to the output of the circuit provided by the inference on the input, and then the technique to protect the model outputs can be considered an extra gate or circuit applied at the end.

for some ciphertext operation $\bigodot$ *that corresponds to a plaintext operation* $\bigotimes$*. The client then performs output reconstruction* MASK.REMOVEALL *by either*

- *doing* $\texttt{agg.fn}(z_0, \ldots, z_{\mathsf{N}-1})$ *over the encrypted responses to get one or more ciphertexts that it can decrypt and then combine into a single* out*, or*
- *decrypting each ciphertext* z_i *using* sk_i *and then performing a plaintext space operation* $\texttt{out} = \texttt{agg.fn}'(y_0 \bigoplus r_0, \ldots, y_{\mathsf{N}-1} \bigoplus r_{\mathsf{N}-1})$*.*

Protocol {MASK.GET, MASK.APPLY, MASK.REMOVEALL} *is said to be a correct if the single output* out *is the same as the aggregation procedure applied in plain to the model outputs* $y_0, \ldots, y_{\mathsf{N}-1}$*.*

Although this definition is cumbersome, we stress that its purpose is straightforward: these random masks are there to hide individual outputs but the system does not work if they cannot be collectively removed.

To instantiate this with output summation and masks that sum to zero, the model owners are given identifiers and placed into pairs by the querying client. The model owners then calculate a random mask per pair, where one of the model owners will add the calculated value to their output and the other model owner will subtract that same value from their model's output. More formally, the aggregation function averages the model responses of a regressor with output space $\{0.0, \ldots, 100.0\}$. We construct pairs of random masks—that are also elements of $\{0.0, \ldots, 100.0\}$—that alternate their sign, since for $z_1 = y_1 + r$ and $z_2 = y_2 - r$ the average (over the integers) of the z_i values is the same as for the y_i values[6]. We cannot use arithmetic mod100 for addition, in the event that one value gets pushed across the boundary by addition and the other does not[7]. We can solve this by extending the function output space in the positive and negative direction, to ensure that wraparound does not occur, This creates a new masked range $\{-100, \ldots, 200\}$ in which the average aggregation will work correctly.

If a different operation than summing is required, for example using Argmax applied to classification models or incorporating contribution evaluation then if it is feasible to construct the corresponding masking scheme then this can be used in our modular framework in a straightforward manner.

In Definition 3.1 the masking of individual model outputs is defined in the presence of a homomorphic encryption scheme, but it is also possible to use the output-hiding ensemble concept in a scenario where model inputs and outputs do not need to be withheld from the model owners. In this case there is no encryption (formally, encryption and decryption are both defined by the identity function) and applying the masks to model outputs is done in plaintext.

3.3 Security Goals

So far we have introduced the building blocks that can be used to provide functionality that we have designed MIOPE to achieve, namely inference to an ensemble of models. We now briefly and informally present the privacy goals that we wish to provide. To complete the threat model we would need to precisely define adversarial capabilities but this is not the goal of our approach: we wish to provide a framework that can use building blocks that are sufficient for a fixed set of assumptions on the adversaries in the system. We will therefore provide in this section an example threat model, and expand on how to deal with stronger adversaries in the discussion in Section 6.

[6]This idea extends to groups of model owners that collectively append randomness such that the sum (and therefore the average) of the output values is the same with or without masking.

[7]If $y_1 = 90, y_2 = 92$ and $r = 20$ then $z_1 = 10, z_2 = 72$ and therefore the average of the z values is 41 rather than the correct value of 91. It's possible to deal with this in plaintext, but in the encrypted domain the entity adding the masks will not be aware of whether wraparound has occurred or not.

For privacy, there are two distinct privacy goals that have to be achieved in input- and output-hiding ensemble inference. Firstly, the model owners performing inference on the input data `qry` should learn no information about the input data or the model output, only that a query has occurred. Secondly, the client should not learn more about the models than the combined aggregated output.

In the instantiation of MIOPE that we describe in Section 4, we target a setting where all participants—the querying client(s) and the model owners—are honest-but-curious. This means that they participate in the sub-protocols as per the definition of those protocols, yet they wish to gain additional information based on the values that they see. Since we use homomorphic encryption we do not strictly require secure channels for communication, therefore parties can freely observe communication between other parties in the network. We assume that parties do not maliciously deviate from the protocol and that there is no collusion between parties.

3.4 MIOPE: A Modular Framework

With this generic setup for output-hiding ensemble inference we have a bottom-up approach where one can choose which algorithms and schemes they wish to instantiate with our system to perform the collective inference. We show this in Fig. 1 where the top-row is an instantiation of the algorithms and schemes used in this paper, which will be clarified in the next Section 4. If the components at the top fulfill the requirements of the underlying components, the system can be used to create an ensemble inference setting with privacy for model inputs and outputs.

4 Output-Hiding Ensemble Inference on Tree-Based Models

In Section 3 we have defined our framework for input- and output-private encrypted inference in ensembles settings. We will now specifically focus on the usage of tree-based models held by multiple model owners in the rest of this paper, but as a reminder our generic framework does not require any specific machine learning architecture and this is just one example instantiation. Machine learning models based on decision trees have shown extremely good performance on tabular data and clinical medicine [ZZC+19, MKV+23].

Tree-based models can consist of a singular tree in the case of decision tree models, or a forest of trees such as in random forest or gradient-boosting models. These models require small modifications to be compatible with encrypted inference to allow for input and output privacy.

4.1 Encrypted Inference on Tree-Based Models

To ensure the encrypted inference works on the tree-based models, we have to create or use a technique to allow homomorphic operations to work on the tree-based models. Generally, tree-based models consist of nodes that contain a decision or threshold that needs to be checked relative to an input feature, to traverse the tree and ultimately obtain a prediction for a given input. However, if a computing party wants to traverse the tree in a secure and encrypted fashion, they cannot take any decision in the tree itself, since this would reveal the path taken (and thus partial information about the input features) and, as the model is owned by the computing party, they would learn the output of the query. This necessitates a way to mask the path being traveled from the computing party, which can be done by transforming the tree-based model into a series of linear and non-linear functions performed over matrices. One of these techniques is the GEMM approach for

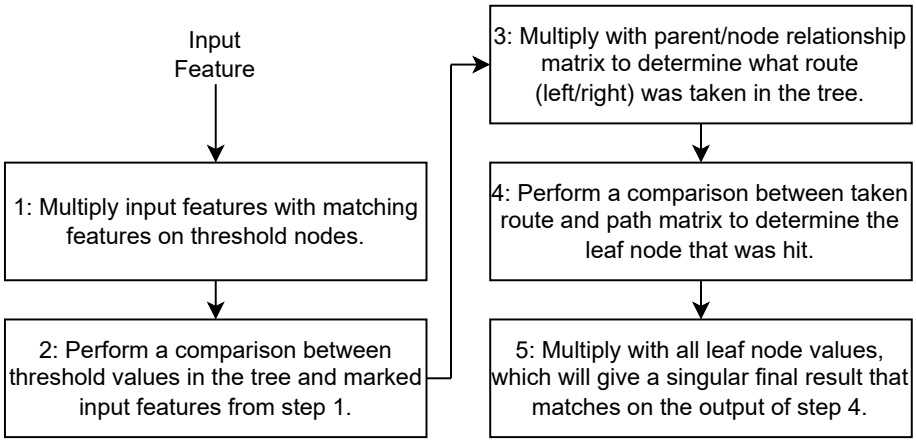

Figure 2: Graphical representation of inference for a tree-based ML model using the GEMM (GEneric Matrix Multiplication) framework, which is used in frameworks such as the `Hummingbird` library [NIS20].

tree-based models which is used to represent the tree-based models in tensor format, as performed by the `Hummingbird` library [NIS20, NSY+20].

4.1.1 ML model to HE Circuit Conversion for Tree-Based models

Concretely, the GEMM inference method transforms the tree-based inference into tensors which are constructed based on the tree model's characteristics. The technique in essence simulates going over every path in the tree with the input data and localizing with tensor operations over encrypted data which final leaf node is hit, yielding only the final leaf value as encrypted output if GEMM is used in combination with homomorphic encryption.

For tree-based models, the GEMM algorithm is a five-step process, see Fig. 2. Let us assume in this description that all steps are performed via homomorphic encryption. First, a multiplication is performed to setup a tensor for feature values which are then compared in the second step with the threshold values of the model. Another multiplication is performed afterward for determining the steps to take to a certain node, after which one more comparison is made to see if a specific path matches. Lastly, a multiplication is performed to obtain the value of the leaf that was matched in the fourth step. In total, the GEMM algorithm consists of 3 tensor multiplications and 2 tensor comparisons. To enable encrypted inference over tensors, multiplication and comparison via the FHE scheme thus need to be supported to prevent the computing party from learning any intermediate outputs.

For our set-up we implement the GEMM algorithm in `OpenFHE` [BAB+22, BBB+22], using it over `Concrete ML` due to its significantly broader scope for configurability and, crucially in our setting to enable model output masking by model owners, its support for public-key operations. The GEMM technique has been implemented in the `Concrete ML` library [Zam22a] as described in academic work by the library's authors [FSB+24], but that library does not directly support PKE.

4.1.2 Implementation in OpenFHE

We require both multiplication and comparison operations in our computation under homomorphic encryption, which can be problematic as most FHE schemes are usually stronger on specific types of operations. Multiplication, for example, is cheap in the integer schemes of FHE, but these schemes usually do not cheaply perform comparisons or approximations. In contrast, binary schemes in FHE have cheaper comparisons but tend to have costlier multiplications. `OpenFHE` allows for implementations in the integer schemes, binary schemes, and also allows for scheme-switching. Integer schemes would require multiplication by approximations of high-degree polynomials to mimic comparisons, and pure binary scheme implementations in `OpenFHE` are less well-documented and lose performance on multiplications. Because machine learning inference on tree-based models has a combination of both non-linear, such as comparisons, and linear operations, we decided to use scheme switching as we have approximately an even split in operations between linear and non-linear.

Our implementation for tree-based encrypted inference starts in the integer scheme CKKS to apply the first multiplication, and scheme-switches to the binary scheme FHEW afterwards to apply the comparison operation. We scheme-switch back to CKKS afterward and repeat for the 3rd and 4th operations in CKKS and FHEW, respectively. The last multiplication with the leaf values is again done in CKKS which is also suitable to allow any additional masking or summation to be performed in CKKS after the GEMM approach has finished.

The GEMM algorithm can be applied to any tree-based model, which allows one to do inference on the features and get a result back per tree. Specifically we stress per tree, as the models that have forests, such as the gradient-boosting and random forest approaches, require a model summation to sum up the leaf values from all the individual trees. For random forests specifically, one also needs to divide by the number of trees used in the summation. Other tree-based models that we do not mention above could require modifications depending on the nature of the tree-based model.

After the GEMM approach and model summation are applied, the final outputs are stored in resultant ciphertext(s) for each model owner. These outputs should be protected in the homomorphic domain using an output model masking technique from section 3.2. The outputs with masks are all sent in encrypted form to the client which should use an appropriate model output summation strategy for the model outputs. The summation method that is used differs slightly for each model type, but also the goal of the model, such as a Classifier or Regressor. For our evaluation, we primarily focused on Regressor models with a simple strategy of summing (i.e., no weighted scores), and the syntax used in the examples reflects this.

4.2 Our Instantiation of MIOPE

The top row of Fig. 1 showcases the instantiation of our approach in terms of schemes and algorithms in the MIOPE architecture. We average over the collected tree-based models, utilize a PRF applied to outputs from the NIST Beacon to generate randomness for the masks, make use of scheme-switching between CKKS and FHEW and use GEMM as a technique to apply Encrypted Inference with. If one wants to support different output model summation techniques, randomness generators, different HE schemes or a different inference algorithm, they can choose to instantiate MIOPE with different components from ours.

Fig. 3 showcases an example instantiation of the MIOPE architecture where data is first encrypted via the keys that were exchanged in the PKE between client-model pairs. The data is encrypted and after that is moved to the model domain where encrypted inference is performed via a function that suits the specific ML model. After the model outputs are

created, the models apply masks generated via the randomness that was acquired prior via the NIST beacon and PRF. The masked outputs are sent back to the client who can decrypt each of the outputs without revealing the exact output of the individual models as they still have masks applied to them. Only after summing the model outputs via model output summation does the client obtain the correct final result.

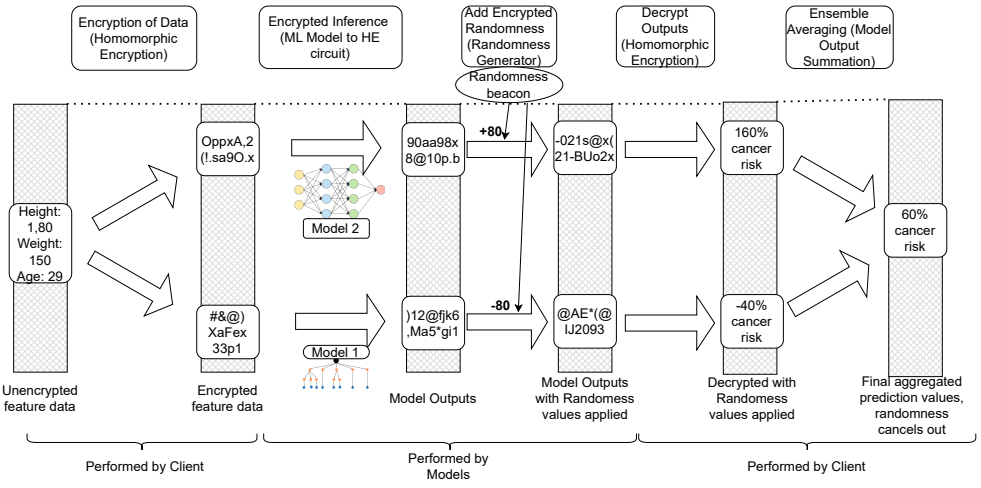

Figure 3: An example of the MIOPE framework to calculate a risk score over two models for cancer given a feature vector. Starting with the client encrypting their feature vector before sending their data to the models to query, which perform encrypted inference and add prepared randomness to their model outputs. The results can be safely decrypted and summed on the client side.

5 Test Case: Output-Hiding Ensemble Inference on California Housing dataset

Our evaluation uses the California housing dataset [PB97], a widely used dataset for demonstrating the effectiveness of regression tasks.[8] The dataset consists of 20,640 entries. We use the dataset to train $\mathsf{N} = X$ Regressor models to determine the median house value based on 8 different features. The features used in this dataset are mostly float values with potentially low values found in features such as AveRooms or high values in the Population. This can have an effect on the comparison operations used in Homomorphic Encryption, which leads us to perform binning and normalization on these values to fit them in our dynamic range. Model training is done using standard techniques within `scikit-learn` and model outputs are normalized to the range $\{0.0, \ldots, 100.0\}$.

To perform output-hiding ensemble inference and compare against centralized learning, we use the train/test split from the centralized learning results and further split the training set according to the amount of model owners we wish to examine. The centralized learning model takes 90% of the available data from the California housing dataset and keeps 10% for testing purposes. Similarly we divide that same 90% of data over our chosen $\mathsf{N} = 4$ amount of models in an IID manner. The models are individually trained over their smaller portion of training data, and the resultant models are queried with the test

[8] Our code for the experimental set-up is available at: https://github.com/kyrianmaat/MIOPE-EPRINT

Model	Tree-Depth	Estimators	Decision Nodes	Learning Rate
Small DT	5	N/A	61	N/A
Medium DT	8	N/A	379	N/A
Large DT	11	N/A	1281	N/A
Small XGB	4	25	775	N/A
Medium XGB	4	100	3058	N/A
Large XGB	4	400	12288	N/A
Small GBR	4	25	765	0.2
Medium GBR	4	100	3020	0.2
Large GBR	4	400	23050	0.1

Figure 4: Parameters of the different tree-based ML models considered for the California Housing dataset. As the decision trees are just a singular model, the depth here was varied while we kept the depth static for the gradient-boosting models that rely on building a forest from shallow decision trees.

data. These inference results are then summed and averaged and used to calculate the Mean Average Error of the output-hiding ensemble inference of MIOPE. The goal of our evaluation is however not to create the best models for the task, but access the viability and accuracy performances of output-hiding ensemble inference when compared to the centralized learning. As such, our training process is not optimized for ideal accuracy.

For this evaluation, we consider Regressors in the form of a Decision Tree (DT) model, a GradientBooster model (GBR) and a eXtreme Gradient Boosting (XGBoost) model[9]. This provides us with a variety of different tree-based models for testing, where the high-depth singular tree of a Decision tree can be compared to the multiple tree evaluation of both gradient boosting models that train by building a forest of shallow trees. We performed a hyperparameter search for the models on tree depth and additionally amount of estimators for the gradient-boosting models. The combinations of parameters used for the small, medium and large versions of these models can be found in Fig. 4. The set-up described in Fig. 4 was used for training the 4 local models of the MIOPE framework, such that we have clusters of 4 similar tree-based models for each combination of parameters. Additionally, we also trained two mixed-model setups to simulate different models being trained in an ensemble, where we trained one mixture with 2 XGB, 1 GBR and 1 DT model and one mixture with 2 XGB and 2 GBR models. Lastly, we trained three XGBoost models using federated learning with 4 clients which mimics the amount of estimators for each of the models by running as many rounds as needed to create enough estimators. We chose to only compare XGB models as this was the most approachable via the Flower framework [BTM+20] but also represented the gradient-boosting model we wanted to compare on most.

To perform inference, the client and model owners use the `OpenFHE` library to setup cryptographic parameters and allow the client to encrypt the data. The data is sent over to the model owners, and the models perform encrypted inference using the GEMM algorithm as described in Section 4.1.1. The model summation for the XGB and GBR models is a simple internal addition of the ciphertext, and the final result is obtained by summation of all model outputs and dividing by the number of model owners. If a Regressor model has a `base_score` for prediction[10], then this has to be added on as an addition in the homomorphic circuit. The random masks can also added via homomorphic addition, which are provided beforehand to the model owners and applied to the encrypted output at inference time.

[9]We tested both AdaBoost and Random Forest models, but the achieved accuracy was considerably worse than the other models in our testing setup. Our encrypted inference pipeline still provides support for these model types.

[10]Specific for tree-based models, they can help steer a prediction to a certain base prediction

Model	Average DecisionNodes	MAE	R2	HE accuracy loss %
Simple LR	N/A	0.528	0.620	< 0.01
Small DT Centralized	61	0.511	0.609	< 0.01
Small DT 4-Split	61	0.479	0.658	< 0.01
Medium DT Centralized	379	0.428	0.701	0.05
Medium DT 4-Split	335	0.409	0.733	0.19
Large DT Centralized	1281	0.393	0.722	N/A
Large DT 4-Split	1117	0.373	0.756	N/A
Small XGB Centralized	775	0.446	0.732	< 0.01
Small XGB 4-Split	771	0.445	0.733	< 0.01
Small XGB FL	358	0.439	0.738	< 0.01
Medium XGB Centralized	3058	0.341	0.824	0.12
Medium XGB 4-Split	2910	0.349	0.817	0.4
Medium XGB FL	1383	0.363	0.805	0.12
Large XGB Centralized	12166	0.313	0.849	0.17
Large XGB 4-Split	11424	0.317	0.843	0.7
Large XGB FL	5500	0.369	0.786	0.15
Small GBR Centralized	765	0.378	0.791	< 0.01
Small GBR 4-Split	753	0.374	0.795	< 0.01
Medium GBR Centralized	3020	0.329	0.835	0.16
Medium GBR 4-Split	2954	0.333	0.831	0.42
Large GBR Centralized	12228	0.314	0.847	0.22
Large GBR 4-Split	11770	0.319	0.841	0.77
Small Mix (DT/GBR/XGB/XGB)	164	0.433	0.732	0.04
Small Mix (GBR/GBR/XGB/XGB)	762	0.399	0.766	0.05
Medium Mix (DT/GBR/XGB/XGB)	2277	0.358	0.805	0.35
Medium Mix (GBR/GBR/XGB/XGB)	2932	0.340	0.824	0.38
Large Mix (DT/GBR/XGB/XGB)	8934	0.331	0.825	N/A
Large Mix (GBR/GBR/XGB/XGB)	11597	0.315	0.844	0.81

Figure 5: Performance of models obtained via centralized learning and via summation over 4 split trained models. Simple LR is a basic baseline model: a linear regressor that only takes into account the first feature. For the split learned models, the average decision nodes had at most a 1 to 2% difference in amount of decision nodes. Only the HE results of models with large Decision Trees could not be measured, as the memory requirement was too stringent on the set-ups.

For the random masks we use the NIST public randomness beacon [oST25] as our source of external randomness (see Section 2.4), where the time to be queried as input to the beacon is defined as the UTC time at which the client made the inference query request. Constructing the normalized random masks from this public random value, the client's identifier and model identifiers is implemented using either AES-CTR or HMAC as its PRF.

5.1 Evaluation

We compare the centralized learning models against the split models from MIOPE by using the Mean Average Error (MAE) as our main test statistic, comparing how it would perform better against a baseline Linear Regression model. A MAE under 0.5 is better than the baseline model, and the smaller the MAE the better. We also show the R2 score, where the closer to 1 the value is the better. We denote the amount of average number of decision nodes the model uses, the MAE associated with that model and the accuracy degradation during inference of using our HE scheme-switching implementation in `OpenFHE`.

As can be seen from Fig. 5, the larger models overall perform better than the starting linear regression algorithm. As the size goes up, the MAE also falls with the best performing models being the GradientBoosting and XGB large models, lowering the MAE to 0.313

	This work	[GJK+18]	FL	Decentralized FL	FL + HE	[XZG24]
Adaptability to changes in used data	●	◐	◐	◐	◐	◐
Local training possible	●	●	○	◐	○	●
Functionality does not require third party	◐	○	○	●	○	○
No significant processing on third party	●	○	◐	●	◐	◐
Short time per query	○	◐	●	●	◐	●
Short time to first query	●	●	○	◐	○	◐
Model Agnosticity	●	●	○	○	○	●

Figure 6: Comparison of MIOPE against other works. MIOPE specifically excels at allowing modular design and changes in the system, but has the most expensive average query time out of all available systems.

from the simple linear regression model's 0.528.

When utilizing MIOPE and performing inference on the newly created split models, we see that the performance is close to the centralized datasets, with minor improvements on the smaller models or slightly lower accuracy. The amount of decision nodes on average for some of these models are higher than the centralized datasets, as the hyperparameter tuning on these individual lower data samples turned out to give different end results.

Comparing the mixed models shows us that they are comparable in accuracy if we omit the decision tree models, as they seem to drag down the average MAE. However, it also means that they can be utilized when lower-resource participants are part of the ensemble. The mixture of GBR and XGB models also seems to outperform an ensemble of only GBR or XGB models, but it is unlikely that this would be reliably reproducible in other experiments.

We ran the federated learning setup for 6, 25 and 100 rounds to obtain 24, 100 and 400 estimators respectively, but we observe that the performance of the FL set-up already peaks around 100 estimators, as the MAE actually drops from the medium to the large set-up. This can partially be attributed to less hyperparameter optimization in FL, but the resultant model also has significantly fewer decision nodes, which might imply that the local models stagnated at some point.

The overall accuracy loss that we observe when utilizing FHE in our approach never goes above 1% of the original accuracy, showcasing that the technique does not add on significant extra accuracy loss. We usually observed a small perturbation due to the nature of HE noise on the accuracy, and the usage of binning in our feature space means that the HE perturbations sometimes push our result in a slightly different bin, which explains the larger than 0.01% HE losses. When we use the MIOPE approach, we see that the error scales relatively linearly with the amount of models that are being used.

We chose to omit results on a split with 8 models trained in Fig. 5 as they showcased worse MAE than the 4-split on all occasions except small and medium decision trees. Splitting more than 8 times on the data set would most likely lead to lower MAE scores and would require different model output summation techniques like weighted scoring, to obtain better results.

6 Discussion and Future Work

We give a small comparison of different works in Fig. 6, showing the difference in the paradigms being used and the trade-offs the systems chose to use.

These trade-offs and advantages, coupled with the added constraint of settings where approaches such as Federated Learning are deemed too invasive on model sharing or where certain legal factors do not allow the usage of federated learning can make Output-Hiding Ensemble Inference a suitable solution. However if other techniques are able to be used in a specific scenario, the downsides of the average inference time might be too significant to offer MIOPE as a suitable alternative.

There are trade-offs made between MIOPE's output-hiding ensemble inference and Federated Learning in terms of the privacy of the models involved. In approaches using Federated Learning the model updates have to be guarded against malicious data owners, but the final model is less vulnerable to model inference attacks than MIOPE. Output-Hiding Ensemble Inference only has to protect the final model against malicious clients that could learn something from the summation, but the system will never have to safeguard the local model updates.

We chose to perform our GEMM inference algorithm via scheme-switching as we had a mixture of linear and non-linear operations, but one could implement the entirety of the algorithm in an integer scheme such as CKKS, or a binary scheme such as FHEW or TFHE, instead. We chose not to explore this direction since it would require very careful noise management and to find parameters that retain acceptable model accuracy, but consider it future work that can be performed to test the accuracy and performance on these methods to compare to scheme switching.

In this work we have implemented our approach using the `OpenFHE` library over the `Concrete ML` library which is designed to support encrypted inference on machine learning models, mostly because PKE is not supported in `Concrete ML` and thus we cannot add any extra data such as our masks to the output of the models. If `Concrete ML` supported Public Key Encryption outright, we would be able to train our models via the `Concrete ML` library or convert the trained models to run in `Concrete ML`. We let `Concrete ML` generate compiled models and cryptoparameters for the client/model pairings, and by using PKE we would add additional operations to add the masks on the output of the models. By adding support for summation on the client with the multiple models, we would be able to support the approach in `Concrete ML`. Support for PKE is currently only supported in the back-end of TFHE-rs, and not directly accessible by purely using the top-level `Concrete ML` library.

The usage of homomorphic encryption inherently will give a longer computation time, which can be crucial for time-critical settings where inference time needs to be below a given threshold. Homomorphic encryption also requires a large amount of memory for the cryptographic parameters to be computed and set, which also need to be transferred to participating parties. If the memory available on the client is limited, then the usage of homomorphic encryption can become troublesome. Other drawbacks of the tree-based encrypted inference techniques are that high-depth trees force a large performance penalty on the computation time of the model. Depending on the models used in for example tree-based learning, this problem can be mitigated somewhat by prioritizing less dense forests in the models that support these. Early exit conditions or trimming the forests can help in lowering the computational requirements.

6.1 Stronger Adversaries and Collusion

Our adversarial setting is all parties being honest-but-curious, and this means that we can tolerate model owners learning the masks of all other model owners. In practice this makes any collusion catastrophic, which is why we provide an instantiation that uses a pair-wise masking technique.

A malicious client can perform arbitrary attacks based on input-output pairs and deviate from the protocol to send out-of-range inputs. To mitigate this threat, MIOPE could be extended either by using a combination of homomorphic encryption scheme and model that ensures that the processing of these inputs always leads to an error output value, or by enforcing a check on the encrypted input, e.g. by asking the client to append a zero-knowledge proof that the input contains an allowed number of features, all within the correct ranges.

A malicious model owner can always withhold the response, provide as masked model output an encryption of an invalid value, or apply an invalid mask. In the mask generation

protocol it could also misbehave, for example by trying to force another model owner to end up with a specific mask (such as 0 in the output space of the model). An adversary that controls multiple model owners can of course do these attacks and potentially have more chance of succeeding. For this reason a tailored approach is needed if usage of MIOPE requires resilience against malicious model owners[11].

In the event that collusion does happen, only combinations of client and model owner collusion would yield information, depending on the model output summation strategy used. If an adversary corrupts model owners, then only in a malicious setting can they influence the outcome of the protocol, as they cannot gain new information on the client due to homomorphic encryption. If an adversary does corrupt a client and model owner(s), then they can cancel out the masks on the outputs in the summation to infer information about the models of the other model owners whose masks cancel out. With a polynomial-time bounded machine, the steps through homomorphic encryption will still take time before a large amount of information of models can be leaked.

If an adversary can control or predict the outputs of the randomness source, then this could have an impact on the privacy leakage within our system. In a setting where the random masks are generated using a symmetric key that is not known to the querying client then an adversary that controls the client and can predict beacon outputs does not gain any advantage in revealing masks (and therefore individual model outputs).

6.2 Future Directions

One future avenue for improvement is to incorporate a mechanism for supporting weighted averages. This could for example allow a client to give a higher weight to models trained with more data points, similar to the concept of *contribution evaluation* in federated learning [XGiAP25]. This is not straightforward since the mask generation process needs to take these weights into account and the client's masked output aggregation process must ensure that the masks really do 'cancel out': the client cannot be the one to apply these weights since this is not possible without some leakage of individual model outputs. We consider it an open problem to efficiently incorporate weighted averages to the mask generation process.

The machine learning models, and particularly the tree-based models with gradient boosting, can support not only numerical or float values for features, but can also utilize categorical data or work on missing values in the features or dataset. However, categorical data and missing feature input values are currently not supported in the GEMM algorithmic approach from the `Hummingbird` library [NIS20, NSY+20] that `Concrete ML` uses, and our approach also does not support categorical or missing data. One way to add support for categorical data would be to change the model to change the singular categorical split on multiple category values to separate threshold splits in the tree, but this is non-trivial and adds significant overhead to every tree.

Supporting a wider array of machine learning model types is also a challenge. Regressors are particularly well suited, while for classifiers it is necessary for the clients in the system to judge how to treat the outputs (masking is straightforward, since classes can be encoded as integers), for example how to proceed if two classes are the result of an equal number of models in the ensemble. Binary classifiers can focus on returning one of the class probabilities and averaging over that, or purely summing up all the predictions and using this as a majority voting. Multi-class classifiers can use a summation function that takes into account the prediction scores for the different labels to sum over. For other models such as ranking or clustering models it might be more challenging to define an appropriate

[11] It is also unclear how realistic this setting of a malicious model owner is, since a system with many clients could potentially disseminate information about the malicious behavior and remove that model owner from the ensemble.

function to interpret and sum the model outputs. Our framework does not appear to be compatible with generative models.

7 Related Literature

7.1 Encrypted Training of Machine Learning models

Encrypted training of machine learning is currently very infeasible in terms of computation and cost, but recent work has shown progress to try to bridge this gap for neural networks [NRPH19, LFFJ20, ZZS+24] and tree-based models [BSCG24]. Encrypted training gives strong advantages and can alleviate the problems of sharing data between model owners, but the training still need to be re-done if models need to be updated or added to the cluster. Our approach still benefits on model updateability at a data owner level, to avoid full re-training at every instance of new data or new model owners joining the setting.

7.2 Federated Learning and Secure Aggregation

There have been many advances in the Federated Learning domain creating a plethora of variations that each tackle different security, privacy and efficiency properties.

Asynchronous Federated Learning (AFL) [CNSR20, XQXG23] tackles the problems regarding communication costs during training for Federated Learning, as whenever updates are send to the global model aggregation will immediately happen on new local models. This means that the training time will be dependent on the weakest link in terms of bottlenecking the model. AFL also improves on the robustness of the system in terms of handling drop-outs, but overall still has a lengthier training time than MIOPE and relies on an aggregation server.

Decentralized Federated Learning (DFL) approaches remove the need for a central aggregation server during the computation [HBJ18, KMA+21, BPS+23], which means no singular entity will be able to observe all the model updates during the training time of the model and the singular entity holding the model at the end is also removed. Due to the nature of FL it still requires re-training for new data or data corruption and the overall training costs more time than MIOPE.

Utilising masks to safeguard updates in FL aggregation was explored in [BIK+17, BBG+20], which define how to do secure aggregation in privacy-preserving manner for machine learning in settings most applicable to Federated Learning. Their focus is primarily on keeping the protocol robust under drop-outs, which is less significant in our setting where we the participants are assumed to stay online, but the masking approach used in secure aggregation is relevant for MIOPE.

7.3 Related subworks to MIOPE

Ensemble learning's [SGU25, SAA+25] training of models and inference is a similar basis as used in MIOPE, but is usually focused on improving the accuracy of the ensemble and not the security and privacy properties of an online ensemble. MIOPE provides input and output privacy for the inference phase of an ensemble.

Giacomelli et al. [GJK+18] and Xu et al. [XZG24] are techniques adjacent to output-hiding ensemble inference either rely on a server to host encrypted models, or focus on Federated Learning in different flavors. The former introduces potential collusion between server and client which needs to be accounted for and has a single node of failure, whilst the latter requires the communication overhead from Federated Learning.

In Giacomelli et al. [GJK+18], the trained models from the model owners are saved in encrypted form on an untrusted server, and by client authentication with the model

owners the client is able to query and aggregate the models. In MIOPE, the aim is to remove this singular node of failure and the assumption that the client and server would not collude.

The work of Xu et al. [XZG24] focus on the federated learning setting but considers heterogeneous models, allowing participants to train models that they consider valuable for their dataset and purpose. They enable local models to train together via a collective voting mechanism on an auxiliary dataset. Samples from this dataset are then labeled according to the votes and aggregated towards all the models if a consensus is reached. Whilst this work also offers opportunities for the participating clients to maintain their own local models, it still requires multiple communication rounds to aggregate votes via a central server.

Another line of work has considered splitting inference query inputs to multiple entities that collaboratively perform inference for a single ML model, including the work of Ma et al. [MLL+19]. This approach follows prior work performing private face recognition using multi-party computation [EFG+09,SZ13], those prior works used dedicated classifiers rather than generic ML models.

References

[AAR+19] Roohallah Alizadehsani, Moloud Abdar, Mohamad Roshanzamir, Abbas Khosravi, Parham M. Kebria, Fahime Khozeimeh, Saeid Nahavandi, Nizal Sarrafzadegan, and U. Rajendra Acharya. Machine learning-based coronary artery disease diagnosis: A comprehensive review. *Comput. Biol. Medicine*, 111, 2019. URL: https://doi.org/10.1016/j.compbiomed.2019.103346, doi:10.1016/J.COMPBIOMED.2019.103346.

[ALR+22] Adi Akavia, Max Leibovich, Yehezkel S. Resheff, Roey Ron, Moni Shahar, and Margarita Vald. Privacy-preserving decision trees training and prediction. *ACM Trans. Priv. Secur.*, 25(3):24:1–24:30, 2022. doi:10.1145/3517197.

[BAB+22] Ahmad Al Badawi, Andreea Alexandru, Jack Bates, Flavio Bergamaschi, David Bruce Cousins, Saroja Erabelli, Nicholas Genise, Shai Halevi, Hamish Hunt, Andrey Kim, Yongwoo Lee, Zeyu Liu, Daniele Micciancio, Carlo Pascoe, Yuriy Polyakov, Ian Quah, Saraswathy R.V., Kurt Rohloff, Jonathan Saylor, Dmitriy Suponitsky, Matthew Triplett, Vinod Vaikuntanathan, and Vincent Zucca. OpenFHE: Open-source fully homomorphic encryption library. GitHub Repository, 2022. https://github.com/openfheorg/openfhe-development. URL: https://github.com/openfheorg/openfhe-development.

[BBB+22] Ahmad Al Badawi, Jack Bates, Flávio Bergamaschi, David Bruce Cousins, Saroja Erabelli, Nicholas Genise, Shai Halevi, Hamish Hunt, Andrey Kim, Yongwoo Lee, Zeyu Liu, Daniele Micciancio, Ian Quah, Yuriy Polyakov, R. V. Saraswathy, Kurt Rohloff, Jonathan Saylor, Dmitriy Suponitsky, Matthew Triplett, Vinod Vaikuntanathan, and Vincent Zucca. OpenFHE: Open-source fully homomorphic encryption library. In Michael Brenner, Anamaria Costache, and Kurt Rohloff, editors, *Proceedings of the 10th Workshop on Encrypted Computing & Applied Homomorphic Cryptography, Los Angeles, CA, USA, 7 November 2022*, pages 53–63. ACM, 2022. doi:10.1145/3560827.3563379.

[BBG+20] James Henry Bell, Kallista A. Bonawitz, Adrià Gascón, Tancrède Lepoint, and Mariana Raykova. Secure single-server aggregation with (poly)logarithmic overhead. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *CCS '20: 2020 ACM SIGSAC Conference on Computer and*

Communications Security, Virtual Event, USA, November 9-13, 2020, pages 1253–1269. ACM, 2020. doi:10.1145/3372297.3417885.

[BCG15] Joseph Bonneau, Jeremy Clark, and Steven Goldfeder. On bitcoin as a public randomness source. *IACR Cryptol. ePrint Arch.*, page 1015, 2015. URL: http://eprint.iacr.org/2015/1015.

[BGGJ20] Christina Boura, Nicolas Gama, Mariya Georgieva, and Dimitar Jetchev. CHIMERA: combining ring-LWE-based fully homomorphic encryption schemes. *J. Math. Cryptol.*, 14(1):316–338, 2020. doi:10.1515/jmc-2019-0026.

[BGV11] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. Fully homomorphic encryption without bootstrapping. *Electron. Colloquium Comput. Complex.*, TR11-111, 2011. URL: https://eccc.weizmann.ac.il/report/2011/111, arXiv:TR11-111.

[BIK+17] Kallista A. Bonawitz, Vladimir Ivanov, Ben Kreuter, Antonio Marcedone, H. Brendan McMahan, Sarvar Patel, Daniel Ramage, Aaron Segal, and Karn Seth. Practical secure aggregation for privacy-preserving machine learning. In Bhavani Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, CCS 2017, Dallas, TX, USA, October 30 - November 03, 2017*, pages 1175–1191. ACM, 2017. doi:10.1145/3133956.3133982.

[BPS+23] Enrique Tomás Martínez Beltrán, Mario Quiles Pérez, Pedro Miguel Sánchez Sánchez, Sergio López Bernal, Gérôme Bovet, Manuel Gil Pérez, Gregorio Martínez Pérez, and Alberto Huertas Celdrán. Decentralized federated learning: Fundamentals, state of the art, frameworks, trends, and challenges. *IEEE Commun. Surv. Tutorials*, 25(4):2983–3013, 2023. doi:10.1109/COMST.2023.3315746.

[Bra12] Zvika Brakerski. Fully homomorphic encryption without modulus switching from classical gapsvp. In Reihaneh Safavi-Naini and Ran Canetti, editors, *Advances in Cryptology - CRYPTO 2012 - 32nd Annual Cryptology Conference, Santa Barbara, CA, USA, August 19-23, 2012. Proceedings*, volume 7417 of *Lecture Notes in Computer Science*, pages 868–886. Springer, 2012. doi:10.1007/978-3-642-32009-5_50.

[BSCG24] Divyanshu Bhardwaj, Sandhya Saravanan, Nishanth Chandran, and Divya Gupta. Securely training decision trees efficiently. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security, CCS 2024, Salt Lake City, UT, USA, October 14-18, 2024*, pages 4673–4687. ACM, 2024. doi:10.1145/3658644.3670268.

[BTM+20] Daniel J Beutel, Taner Topal, Akhil Mathur, Xinchi Qiu, Javier Fernandez-Marques, Yan Gao, Lorenzo Sani, Hei Li Kwing, Titouan Parcollet, Pedro PB de Gusmão, and Nicholas D Lane. Flower: A friendly federated learning research framework. *arXiv preprint arXiv:2007.14390*, 2020.

[CDPP22] Kelong Cong, Debajyoti Das, Jeongeun Park, and Hilder V. L. Pereira. Sortinghat: Efficient private decision tree evaluation via homomorphic encryption and transciphering. In Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi, editors, *Proceedings of the 2022 ACM SIGSAC Conference on Computer*

and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7-11, 2022, pages 563–577. ACM, 2022. doi:10.1145/3548606.3560702.

[CGGI16] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds. In Jung Hee Cheon and Tsuyoshi Takagi, editors, *Advances in Cryptology - ASIACRYPT 2016 - 22nd International Conference on the Theory and Application of Cryptology and Information Security, Hanoi, Vietnam, December 4-8, 2016, Proceedings, Part I*, volume 10031 of *Lecture Notes in Computer Science*, pages 3–33, 2016. doi:10.1007/978-3-662-53887-6_1.

[CGGI20] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. TFHE: fast fully homomorphic encryption over the torus. *J. Cryptol.*, 33(1):34–91, 2020. doi:10.1007/s00145-019-09319-x.

[CH10] Jeremy Clark and Urs Hengartner. On the use of financial data as a random beacon. In Douglas W. Jones, Jean-Jacques Quisquater, and Eric Rescorla, editors, *2010 Electronic Voting Technology Workshop / Workshop on Trustworthy Elections, EVT/WOTE '10, Washington, D.C., USA, August 9-10, 2010*. USENIX Association, 2010. URL: https://www.usenix.org/conference/evtwote-10/use-financial-data-random-beacon.

[CKKS17] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yong Soo Song. Homomorphic encryption for arithmetic of approximate numbers. In Tsuyoshi Takagi and Thomas Peyrin, editors, *Advances in Cryptology - ASIACRYPT 2017 - 23rd International Conference on the Theory and Applications of Cryptology and Information Security, Hong Kong, China, December 3-7, 2017, Proceedings, Part I*, volume 10624 of *Lecture Notes in Computer Science*, pages 409–437. Springer, 2017. doi:10.1007/978-3-319-70694-8_15.

[CNSR20] Yujing Chen, Yue Ning, Martin Slawski, and Huzefa Rangwala. Asynchronous online federated learning for edge devices with non-iid data. In Xintao Wu, Chris Jermaine, Li Xiong, Xiaohua Hu, Olivera Kotevska, Siyuan Lu, Weija Xu, Srinivas Aluru, Chengxiang Zhai, Eyhab Al-Masri, Zhiyuan Chen, and Jeff Saltz, editors, *2020 IEEE International Conference on Big Data (IEEE BigData 2020), Atlanta, GA, USA, December 10-13, 2020*, pages 15–24. IEEE, 2020. URL: https://doi.org/10.1109/BigData50022.2020.9378161, doi:10.1109/BIGDATA50022.2020.9378161.

[DDS+09] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR 2009), 20-25 June 2009, Miami, Florida, USA*, pages 248–255. IEEE Computer Society, 2009. doi:10.1109/CVPR.2009.5206848.

[DM15] Léo Ducas and Daniele Micciancio. FHEW: bootstrapping homomorphic encryption in less than a second. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology - EUROCRYPT 2015 - 34th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Sofia, Bulgaria, April 26-30, 2015, Proceedings, Part I*, volume 9056 of *Lecture Notes in Computer Science*, pages 617–640. Springer, 2015. doi:10.1007/978-3-662-46800-5_24.

[Dwo06] Cynthia Dwork. Differential privacy. In Michele Bugliesi, Bart Preneel, Vladimiro Sassone, and Ingo Wegener, editors, *Automata, Languages and*

Programming, 33rd International Colloquium, ICALP 2006, Venice, Italy, July 10-14, 2006, Proceedings, Part II, volume 4052 of *Lecture Notes in Computer Science*, pages 1–12. Springer, 2006. doi:10.1007/11787006_1.

[EFG+09] Zekeriya Erkin, Martin Franz, Jorge Guajardo, Stefan Katzenbeisser, Inald Lagendijk, and Tomas Toft. Privacy-preserving face recognition. In Ian Goldberg and Mikhail J. Atallah, editors, *Privacy Enhancing Technologies, 9th International Symposium, PETS 2009, Seattle, WA, USA, August 5-7, 2009. Proceedings*, volume 5672 of *Lecture Notes in Computer Science*, pages 235–253. Springer, 2009. doi:10.1007/978-3-642-03168-7_14.

[Efr92] Bradley Efron. Bootstrap methods: another look at the jackknife. In *Breakthroughs in statistics: Methodology and distribution*, pages 569–593. Springer, 1992.

[FSB+24] Jordan Frery, Andrei Stoian, Roman Bredehoft, Luis Montero, Celia Kherfallah, Benoit Chevallier-Mames, and Arthur Meyre. Privacy-preserving tree-based inference with TFHE. In Samia Bouzefrane, Soumya Banerjee, Fabrice Mourlin, Selma Boumerdassi, and Éric Renault, editors, *Mobile, Secure, and Programmable Networking*, pages 139–156, Cham, 2024. Springer Nature Switzerland.

[FV12] Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. *IACR Cryptol. ePrint Arch.*, page 144, 2012. URL: http://eprint.iacr.org/2012/144.

[GBDM20] Jonas Geiping, Hartmut Bauermeister, Hannah Dröge, and Michael Moeller. Inverting gradients - how easy is it to break privacy in federated learning? In Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin, editors, *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020. URL: https://proceedings.neurips.cc/paper/2020/hash/c4ede56bbd98819ae6112b20ac6bf145-Abstract.html.

[Gen09] Craig Gentry. *A fully homomorphic encryption scheme*. PhD thesis, Stanford University, USA, 2009. URL: https://searchworks.stanford.edu/view/8493082.

[GJK+18] Irene Giacomelli, Somesh Jha, Ross Kleiman, David Page, and Kyonghwan Yoon. Privacy-preserving collaborative prediction using random forests, 2018. URL: http://arxiv.org/abs/1811.08695, arXiv:1811.08695.

[GLMS23] Alejandro Guerra-Manzanares, L. Julian Lechuga Lopez, Michail Maniatakos, and Farah E. Shamout. Privacy-preserving machine learning for healthcare: Open challenges and future perspectives. In Hao Chen and Luyang Luo, editors, *Trustworthy Machine Learning for Healthcare - First International Workshop, TML4H 2023, Virtual Event, May 4, 2023, Proceedings*, volume 13932 of *Lecture Notes in Computer Science*, pages 25–40. Springer, 2023. doi:10.1007/978-3-031-39539-0_3.

[GST+99] Todd R Golub, Donna K Slonim, Pablo Tamayo, Christine Huard, Michelle Gaasenbeek, Jill P Mesirov, Hilary Coller, Mignon L Loh, James R Downing, Mark A Caligiuri, et al. Molecular classification of cancer: class discovery and class prediction by gene expression monitoring. *science*, 286(5439):531–537, 1999.

[HBJ18] Lie He, An Bian, and Martin Jaggi. COLA: decentralized linear learning. In Samy Bengio, Hanna M. Wallach, Hugo Larochelle, Kristen Grauman, Nicolò Cesa-Bianchi, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 31: Annual Conference on Neural Information Processing Systems 2018, NeurIPS 2018, December 3-8, 2018, Montréal, Canada*, pages 4541–4551, 2018. URL: https://proceedings.neurips.cc/paper/2018/hash/05a70454516ecd9194c293b0e415777f-Abstract.html.

[KMA+21] Peter Kairouz, H. Brendan McMahan, Brendan Avent, Aurélien Bellet, Mehdi Bennis, Arjun Nitin Bhagoji, Kallista A. Bonawitz, Zachary Charles, Graham Cormode, Rachel Cummings, Rafael G. L. D'Oliveira, Hubert Eichner, Salim El Rouayheb, David Evans, Josh Gardner, Zachary Garrett, Adrià Gascón, Badih Ghazi, Phillip B. Gibbons, Marco Gruteser, Zaïd Harchaoui, Chaoyang He, Lie He, Zhouyuan Huo, Ben Hutchinson, Justin Hsu, Martin Jaggi, Tara Javidi, Gauri Joshi, Mikhail Khodak, Jakub Konečný, Aleksandra Korolova, Farinaz Koushanfar, Sanmi Koyejo, Tancrède Lepoint, Yang Liu, Prateek Mittal, Mehryar Mohri, Richard Nock, Ayfer Özgür, Rasmus Pagh, Hang Qi, Daniel Ramage, Ramesh Raskar, Mariana Raykova, Dawn Song, Weikang Song, Sebastian U. Stich, Ziteng Sun, Ananda Theertha Suresh, Florian Tramèr, Praneeth Vepakomma, Jianyu Wang, Li Xiong, Zheng Xu, Qiang Yang, Felix X. Yu, Han Yu, and Sen Zhao. Advances and open problems in federated learning. *Found. Trends Mach. Learn.*, 14(1-2):1–210, 2021. doi:10.1561/2200000083.

[KMR15] Jakub Konečný, Brendan McMahan, and Daniel Ramage. Federated optimization: Distributed optimization beyond the datacenter. *CoRR*, abs/1511.03575, 2015. URL: http://arxiv.org/abs/1511.03575, arXiv:1511.03575.

[KMY+16] Jakub Konečný, H. Brendan McMahan, Felix X. Yu, Peter Richtárik, Ananda Theertha Suresh, and Dave Bacon. Federated learning: Strategies for improving communication efficiency. *CoRR*, abs/1610.05492, 2016. URL: http://arxiv.org/abs/1610.05492, arXiv:1610.05492.

[KNL+19] Ágnes Kiss, Masoud Naderpour, Jian Liu, N. Asokan, and Thomas Schneider. Sok: Modular and efficient private decision tree evaluation. *Proc. Priv. Enhancing Technol.*, 2019(2):187–208, 2019. URL: https://doi.org/10.2478/popets-2019-0026, doi:10.2478/POPETS-2019-0026.

[KTBG20] Dan Kondratyuk, Mingxing Tan, Matthew Brown, and Boqing Gong. When ensembling smaller models is more efficient than single large models. *CoRR*, abs/2005.00570, 2020. URL: https://arxiv.org/abs/2005.00570, arXiv:2005.00570.

[LFFJ20] Qian Lou, Bo Feng, Geoffrey Charles Fox, and Lei Jiang. Glyph: Fast and accurately training deep neural networks on encrypted data. In Hugo Larochelle, Marc'Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin, editors, *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020. URL: https://proceedings.neurips.cc/paper/2020/hash/685ac8cadc1be5ac98da9556bc1c8d9e-Abstract.html.

[LHH+21] Wen-jie Lu, Zhicong Huang, Cheng Hong, Yiping Ma, and Hunter Qu. PEGASUS: bridging polynomial and non-polynomial evaluations in homomorphic encryption. In *42nd IEEE Symposium on Security and Privacy, SP 2021,*

San Francisco, CA, USA, 24-27 May 2021, pages 1057–1073. IEEE, 2021. doi:10.1109/SP40001.2021.00043.

[MKV+23] Duncan McElfresh, Sujay Khandagale, Jonathan Valverde, Vishak Prasad C, Ganesh Ramakrishnan, Micah Goldblum, and Colin White. When do neural nets outperform boosted trees on tabular data? *Advances in Neural Information Processing Systems*, 36:76336–76369, 2023.

[MLL+19] Zhuo Ma, Yang Liu, Ximeng Liu, Jianfeng Ma, and Kui Ren. Lightweight privacy-preserving ensemble classification for face recognition. *IEEE Internet Things J.*, 6(3):5778–5790, 2019. doi:10.1109/JIOT.2019.2905555.

[MNLK23] Rasoul Akhavan Mahdavi, Haoyan Ni, Dimitry Linkov, and Florian Kerschbaum. Level up: Private non-interactive decision tree evaluation using levelled homomorphic encryption. In Weizhi Meng, Christian Damsgaard Jensen, Cas Cremers, and Engin Kirda, editors, *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS 2023, Copenhagen, Denmark, November 26-30, 2023*, pages 2945–2958. ACM, 2023. doi:10.1145/3576915.3623095.

[MWCB24] Zoltán Ádám Mann, Christian Weinert, Daphnee Chabal, and Joppe W Bos. Towards practical secure neural network inference: the journey so far and the road ahead. *ACM Computing Surveys*, 56(5), 2024.

[NIS20] Supun Nakandala, Matteo Interlandi, and Karla Saur. Hummingbird. GitHub Repository, 2020. https://github.com/microsoft/hummingbird. URL: https://github.com/microsoft/hummingbird.

[NRPH19] Karthik Nandakumar, Nalini K. Ratha, Sharath Pankanti, and Shai Halevi. Towards deep neural network training on encrypted data. In *IEEE Conference on Computer Vision and Pattern Recognition Workshops, CVPR Workshops 2019, Long Beach, CA, USA, June 16-20, 2019*, pages 40–48. Computer Vision Foundation / IEEE, 2019. URL: http://openaccess.thecvf.com/content_CVPRW_2019/html/CV-COPS/Nandakumar_Towards_Deep_Neural_Network_Training_on_Encrypted_Data_CVPRW_2019_paper.html, doi:10.1109/CVPRW.2019.00011.

[NSY+20] Supun Nakandala, Karla Saur, Gyeong-In Yu, Konstantinos Karanasos, Carlo Curino, Markus Weimer, and Matteo Interlandi. A tensor compiler for unified machine learning prediction serving. In *14th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2020, Virtual Event, November 4-6, 2020*, pages 899–917. USENIX Association, 2020. URL: https://www.usenix.org/conference/osdi20/presentation/nakandala.

[oST25] National Institute of Standards and Technology. Interoperable randomness beacons: Beacon 2.0. https://csrc.nist.gov/projects/interoperable-randomness-beacons/beacon-20, 2025. Accessed: 2025-08-13.

[PB97] R. Kelley Pace and Ronald Barry. Sparse spatial autoregressions. *Statistics and Probability Letters*, 33:291–297, 1997.

[Rab83] Michael O. Rabin. Transaction protection by beacons. *J. Comput. Syst. Sci.*, 27(2):256–267, 1983. doi:10.1016/0022-0000(83)90042-9.

[SAA+25] Enoch Sakyi-Yeboah, Edmund Fosu Agyemang, Vincent Agbenyeavu, Akua Osei-Nkwantabisa, Priscilla Kissi-Appiah, Lateef Moshood, Lawrence Agbota, and Ezekiel N. N. Nortey. Heart disease prediction using ensemble tree

algorithms: A supervised learning perspective. *Appl. Comput. Intell. Soft Comput.*, 2025(1), 2025. URL: https://doi.org/10.1155/acis/1989813, doi:10.1155/ACIS/1989813.

[SGU25] Aparna Saju, Shivaprasad Gundibail, and Ujjwal. Cloud-enabled predictive modeling of mental health using ensemble machine learning models and AES-256 security. *IEEE Access*, 13:152948–152961, 2025. doi:10.1109/ACCESS.2025.3601896.

[SSSS17] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. Membership inference attacks against machine learning models. In *2017 IEEE Symposium on Security and Privacy, SP 2017, San Jose, CA, USA, May 22-26, 2017*, pages 3–18. IEEE Computer Society, 2017. doi:10.1109/SP.2017.41.

[SZ13] Thomas Schneider and Michael Zohner. GMW vs. Yao? efficient secure two-party computation with low depth circuits. In Ahmad-Reza Sadeghi, editor, *Financial Cryptography and Data Security - 17th International Conference, FC 2013, Okinawa, Japan, April 1-5, 2013, Revised Selected Papers*, volume 7859 of *Lecture Notes in Computer Science*, pages 275–292. Springer, 2013. doi:10.1007/978-3-642-39884-1_23.

[TBK20] Anselme Tueno, Yordan Boev, and Florian Kerschbaum. Non-interactive private decision tree evaluation. In Anoop Singhal and Jaideep Vaidya, editors, *Data and Applications Security and Privacy XXXIV - 34th Annual IFIP WG 11.3 Conference, DBSec 2020, Regensburg, Germany, June 25-26, 2020, Proceedings*, volume 12122 of *Lecture Notes in Computer Science*, pages 174–194. Springer, 2020. doi:10.1007/978-3-030-49669-2_10.

[TKK19] Anselme Tueno, Florian Kerschbaum, and Stefan Katzenbeisser. Private evaluation of decision trees using sublinear cost. *Proc. Priv. Enhancing Technol.*, 2019(1):266–286, 2019. URL: https://doi.org/10.2478/popets-2019-0015, doi:10.2478/POPETS-2019-0015.

[TMZC17] Raymond K. H. Tai, Jack P. K. Ma, Yongjun Zhao, and Sherman S. M. Chow. Privacy-preserving decision trees evaluation via linear functions. In Simon N. Foley, Dieter Gollmann, and Einar Snekkenes, editors, *Computer Security - ESORICS 2017 - 22nd European Symposium on Research in Computer Security, Oslo, Norway, September 11-15, 2017, Proceedings, Part II*, volume 10493 of *Lecture Notes in Computer Science*, pages 494–512. Springer, 2017. doi:10.1007/978-3-319-66399-9_27.

[WFNL16] David J. Wu, Tony Feng, Michael Naehrig, and Kristin E. Lauter. Privately evaluating decision trees and random forests. *Proc. Priv. Enhancing Technol.*, 2016(4):335–355, 2016. URL: https://doi.org/10.1515/popets-2016-0043, doi:10.1515/POPETS-2016-0043.

[XGiAP25] Marvin Xhemrishi, Alexandre Graell i Amat, and Balazs Pejo. Detect & score: Privacy-preserving misbehavior detection and contribution evaluation in federated learning. In *Proceedings of the International Workshop on Secure and Efficient Federated Learning*, FL-AsiaCCS '25, New York, NY, USA, 2025. Association for Computing Machinery. doi:10.1145/3709023.3737692.

[XQXG23] Chenhao Xu, Youyang Qu, Yong Xiang, and Longxiang Gao. Asynchronous federated learning on heterogeneous devices: A survey. *Comput. Sci. Rev.*, 50:100595, 2023. URL: https://doi.org/10.1016/j.cosrev.2023.100595, doi:10.1016/J.COSREV.2023.100595.

[XZG24] Yukai Xu, Jingfeng Zhang, and Yujie Gu. Privacy-preserving heterogeneous federated learning for sensitive healthcare data, 2024. URL: https://doi.org/10.48550/arXiv.2406.10563, arXiv:2406.10563, doi:10.48550/ARXIV.2406.10563.

[YZH22] Xuefei Yin, Yanming Zhu, and Jiankun Hu. A comprehensive survey of privacy-preserving federated learning: A taxonomy, review, and future directions. *ACM Comput. Surv.*, 54(6):131:1–131:36, 2022. doi:10.1145/3460427.

[Zam22a] Zama. Concrete ML: a privacy-preserving machine learning library using fully homomorphic encryption for data scientists, 2022. https://github.com/zama-ai/concrete-ml.

[Zam22b] Zama. Concrete: TFHE Compiler that converts Python programs into FHE equivalent, 2022. https://github.com/zama-ai/concrete.

[ZDW13] Yuchen Zhang, John C. Duchi, and Martin J. Wainwright. Communication-efficient algorithms for statistical optimization. *J. Mach. Learn. Res.*, 14(1):3321–3363, 2013. URL: https://dl.acm.org/doi/10.5555/2567709.2567769, doi:10.5555/2567709.2567769.

[ZLL+18] Yue Zhao, Meng Li, Liangzhen Lai, Naveen Suda, Damon Civin, and Vikas Chandra. Federated learning with non-iid data. *CoRR*, abs/1806.00582, 2018. URL: http://arxiv.org/abs/1806.00582, arXiv:1806.00582.

[ZLZ23] Haili Zhang, Zhaobo Liu, and Guohua Zou. Least squares model averaging for distributed data. *J. Mach. Learn. Res.*, 24:215:1–215:59, 2023. URL: https://jmlr.org/papers/v24/22-0511.html.

[ZYWL25] Zhiyuan Zhai, Xiaojun Yuan, Xin Wang, and Geoffrey Ye Li. Decentralized federated learning with distributed aggregation weight optimization. *arXiv preprint arXiv:2511.03284*, 2025.

[ZZC+19] Zhongheng Zhang, Yiming Zhao, Aran Canes, Dan Steinberg, Olga Lyashevska, et al. Predictive analytics with gradient boosting in clinical medicine. *Annals of translational medicine*, 7(7):152, 2019.

[ZZS+24] Yancheng Zhang, Mengxin Zheng, Yuzhang Shang, Xun Chen, and Qian Lou. Heprune: Fast private training of deep neural networks with encrypted data pruning. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang, editors, *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024. URL: http://papers.nips.cc/paper_files/paper/2024/hash/5b26b9e634ba10f6c51c6db7365c4c28-Abstract-Conference.html.